IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 21 September 2024, accepted 6 October 2024, date of publication 11 October 2024, date of current version 22 October 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3478833

RESEARCH ARTICLE

The Smart Product Backlog: A Classification Model of User Stories

MAURICIO GAONA-CUEVAS[1], **VÍCTOR BUCHELI GUERRERO**[1], **AND FREDY H. VERA-RIVERA**[2]

[1]Escuela de Ingeniería de Sistemas y Computación, Facultad de Ingeniería, Universidad del Valle, Cali 760042, Colombia
[2]Grupo de Investigación en Inteligencia Artificial (GIA), Departamento de Sistemas, Universidad Francisco de Paula Santander, Cúcuta 546552, Colombia

Corresponding author: Fredy H. Vera-Rivera (fredyhumbertovera@ufps.edu.co)

ABSTRACT In agile software development processes, user stories (US) had been used to specify application functionalities from the users' perspective. For intelligent applications leveraging artificial intelligence (AI), the Smart Product Backlog (SPB) has included both AI-implementable and non-AI functionalities. This paper had proposed a model employing supervised machine learning techniques to classify US descriptions based on their technical feasibility for AI implementation. This model had aimed to assist in constructing smart product backlogs (SPB). Classifying US in agile development, particularly with AI, had been a labor-intensive process demanding significant time and expert involvement. Additionally, the lack of a dedicated dataset for this task had limited the applicability of automated methods. This study addressed this challenge by having experts classify the Mendeley US dataset using binary classification (AI and non-AI). The proposal had involved developing an automatic classification model to process US descriptions and distinguish those suitable for AI implementation. This model had leveraged advanced text processing techniques to refine the textual features of the US. Additionally, it had employed three binary classification techniques: logistic regression, k-nearest neighbors (k-NN), and support vector machines (SVM). The model's performance had been evaluated using metrics such as accuracy, loss (Log-Likelihood Loss), precision, recall, F1 score, area under the ROC curve, and specificity to identify the best-performing technique in binary classification. Logistic regression and SVM models had demonstrated high accuracy, with scores of 0.748 and 0.740, respectively. These results had highlighted the potential of an automated tool for recommending US feasible for AI development, thereby supporting decision-making in agile software projects.

INDEX TERMS User stories classification, smart product backlog, smart user story identifier, requirements elicitation, agile software development (ASD), machine learning, supervised learning classifiers.

I. INTRODUCTION

Agile software development processes use user stories (US) to specify the application's functionalities, describing the desired functionality from the users' perspective. Additionally, US clearly express the value that a requirement brings to an application. The Product Backlog includes the identified features for an application, which are represented by US [1].

The product backlog (PB) is a prioritized list of all tasks, features, and requirements necessary to complete a development project. The use of AI in software application implementation has enabled the creation of so-called intelligent applications. In these intelligent applications, the PB can contain both functionalities that will be implemented with AI and those that will not, and this type of PB is referred to as a Smart Product Backlog (SPB).

In the context of intelligent applications, which leverage artificial intelligence (AI) to offer innovative functionalities, the SPB allows development teams to have a complete view of the functionalities and capabilities that AI can provide to the development project.

Currently, building the SPB is a manual process, where the time required for this process, the need for AI experts, and

The associate editor coordinating the review of this manuscript and approving it for publication was Porfirio Tramontana.

© 2024 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

its subjectivity make the construction of the SPB a challenge in itself. The manual approach demands time and consumes valuable resources, lacks standardization, presents scalability limitations, and is additionally prone to human errors and biases. This could lead to rework, additional costs, and suboptimal system performance [2]. Therefore, assisting the decision-making process in building the SPB is key to the development of intelligent applications and allows teams to strategically leverage the advantages of AI.

To properly classify and prioritize US in software development, it is crucial to understand the specific characteristics and defined functionality in each US, as well as to evaluate their potential for improvement through the implementation of AI techniques. US suitable for AI typically include functionalities such as automated personalization, predictive analysis, and continuous improvement based on machine learning from user-generated data. On the other hand, transactional US focus on data management, direct user interaction, and standard transaction processing. Understanding that there is a distinction between US implementable with AI and those that are not, this work proposes an automatic US classifier. It automatically identifies patterns associated with the complexity of each US, recognizing US implementable with AI and those that are not. In the proposed model, the machine learning task is performed on the text of the US description in English.

This work proposes a binary classification model for US developable with AI. For the model development, different representations of user stories were evaluated, such as TF-IDF (Term Frequency-Inverse Document Frequency) and pre-trained language vectors (Word Vectors and character vectors). Three techniques were compared for building the classification model: logistic classifier, nearest neighbors, and support vector machine. For each one, the hyperparameters were optimized and standard metrics in the construction of a classification model were evaluated, such as accuracy, precision, recall, f1-score, and area under the ROC curve.

In this research, the Connextra format was chosen due to its widespread adoption in agile methodologies and its clear, consistent structure (''As [role], I want [functionality], so that [benefit]''). This uniformity minimizes variability in text formulation, simplifying the extraction of key features and the identification of relevant patterns in textual data. Consequently, it enhances both the accuracy and generalization capabilities of the classification model.

A dataset was constructed for this project, containing 1680 US, previously classified by AI and software development experts. The evaluated classification model results show that the accuracies are above 0.74, the precisions are close to 0.76, and the area under the ROC curve is greater than 0.8. The results are promising and demonstrate the feasibility of building a tool for the automatic recommendation of US developable with AI.

This document is structured into sections that include related works (section II), methodology (section III), results (section IV), discussion (section V), and conclusions (section VI).

II. RELATED WORKS

To identify related works, a literature review was conducted in the scientific databases: Scopus, IEEE, Google Scholar, and ACM. The guidelines proposed by Kitchenham [3] were followed for this purpose. Thus, after reviewing and selecting documents, the following related articles were obtained:

Kaur and Kaur [2] highlight the potential of Artificial Intelligence (AI) to automate the identification and classification of software requirements into functional or non-functional requirements, especially in large-scale and complex projects. The study reviewed 60 research works that employ automatic techniques, concluding that transfer learning based on convolutional neural networks (CNN) and support vector machines (SVM) achieve the best results. However, the study also notes that the applicability of these techniques in real and complex environments is not yet fully proven. This suggests the need for greater collaboration between requirements engineering and AI to overcome the challenges faced by automated systems in the real world.

Subedi et al. [4] address the topic of automating the classification of US in agile methodologies using supervised learning classifiers. The study focuses on the ''Testable'' and ''Valuable'' criteria from the INVEST checklist and explores the impact of data balancing techniques on improving classification metrics.

Ma et al. [5] address the recognition and automatic segmentation of compound US through semantic analysis. The research focuses on improving the understanding of US in agile development, using a system based on HanLP to identify and segment these stories efficiently.

Dos Santos et al. [1] address the automatic generation of US in agile software development. They use N-gram representation approaches with linguistic heuristics and the GPT-3 model, evaluating both methods using BLEU, ROUGE-N, and BERT Score metrics.

Kumar et al. [6] highlight the relevance of non-functional requirements (NFR) in requirements engineering for software development. They propose an approach to classify NFR based on the importance of US in Agile Software Development (ASD) using machine learning models. The results show that random forest and XGBoost classifiers achieved an accuracy of 96.39%, followed by the decision tree classifier with an accuracy of 96.13%. They are compared with other models like K-nearest neighbors, linear discriminant analysis, support vector classifier, logistic regression, and naïve Bayes.

Tasnim et al. [7] address the challenges of the manual classification process in Software Requirements Specification (SRS), highlighting subjectivity, time required, and the possibility of errors. They explore automation in distinguishing between functional and non-functional requirements using a set of machine learning classifiers. With test data, empirical results show that the proposed ensemble method achieves 98% accuracy in the binary classification

of Functional Requirements (FR) and Non-Functional Requirements (NFR). Additionally, hyperparameter optimization improves its performance, highlighting its utility for improving the accuracy and consistency of software requirements analysis.

Levy et al. [8] present the instability metric to evaluate US in agile development, measuring the number of changes made after their assignment. They propose a method to create predictive models that identify US with high instability before assigning them to developers. The models were evaluated in various open-source projects and one commercial project, outperforming the performance of baseline models. Four classifiers were examined: logistic regression, KNN, Naive Bayes, and SVM. SVM and logistic regression did not outperform a random classifier, while KNN achieved the highest accuracy (86%).

Raharjana et al. [9] A systematic literature review to capture the current state-of-the-art of NLP research on user stories. The category of studies mostly performed is key abstraction identification of user stories and generation of models or artifacts from user stories. Part-of-Speech (POS) tags are the most widely used NLP techniques, but semantic approaches (e.g., vector space models and machine learning) are starting to gain a place.

Siahaan et al. [10] propose a technique to extract US from natural language in online news. Three key aspects are analyzed: who (interested actor), what (actor's desire), and why (reason for the desire). The method combines linguistic techniques to identify named entities, grammatical dependencies, and domain-specific vocabulary. The results show that online news can generate objective or subjective requirements. They improved accuracy in identifying actors and extracting information related to the software. This research demonstrates that non-software-related sources, such as news, can contribute to requirements elicitation.

Rahimi et al. [11] evaluate the effectiveness of three deep learning (DL) ensemble approaches to classify software requirements (SR) into functional (FR) and non-functional (NFR), and then sub-classify them into 17 classes. The proposed classification systems, trained and evaluated on the PROMISE dataset, show competitive performance compared to state-of-the-art methods. The two-phase classification system, which classifies SR into FR/NFR in the first phase and then into 17 subclasses in the second, achieves a remarkable 95.7% accuracy in the binary classification phase and 93.4% in the multiclass classification phase.

Rahman and Zhu [12] developed a tool called "GeneUS" using GPT-4.0 to automatically create user stories from requirements documents, which are the result of the requirements engineering (RE) phase. The result is provided in JSON format, allowing easy integration with popular project management tools. Analyzing requirements documents requires significant effort and multiple meetings with stakeholders. We believe that automating this process will reduce the additional burden on software engineers and increase productivity, as they will be able to dedicate their time to other priority tasks.

The literature review shows that there is a wide variety of approaches used for the classification of US or related to the requirements elicitation process (see Table 1). However, there are no specific works on the proposed topic in this research. On the other hand, some machine learning techniques stand out for their frequency and effectiveness. Among them, the most employed are logistic regression, K-Nearest Neighbors (KNN), and Support Vector Machines (SVM). These consistently show good results in multiple similar studies.

In Table 1, it is observed that logistic regression is used for its simplicity and ability to handle binary classification problems effectively. KNN is also widely employed due to its robustness in classifying complex data and its ability to handle large feature spaces. Likewise, there is growing interest in advanced techniques and hybrid approaches for the classification of User Stories (US). Ensemble models like random forest and XGBoost, which combine multiple classifiers, have shown improvements in prediction accuracy and robustness. Additionally, data balancing techniques are effective in improving classification metrics by handling class distribution disproportion. Moreover, methods based on semantic analysis and natural language processing (NLP) have been explored. Tools like HanLP and N-gram-based techniques, along with large language models (LLM) like GPT-3 and GPT-4, are used to improve the understanding and segmentation of US, allowing the capture of complex semantic relationships and improving classification accuracy.

In summary, the most used and effective methods for US classification include logistic regression, KNN, and SVM. These methods will be used in this work to build a user story classification model. It is noted that a search was conducted in various academic sources and specialized databases, but no articles specifically addressing the classification of user stories for implementation with artificial intelligence techniques were found. This work is presented as a first approach to the problem of building an automatic classification model for US implementable with AI.

III. METHODOLOGY

This section formulates the questions that drove the research and identifies the sources of information used. A detailed description of the data set used is provided, and the methods used to build a classification model are explained. In addition, the metrics to evaluate the results are described, and the model's architecture is presented.

A. RESEARCH QUESTIONS

The researchers have utilized mathematical, statistical, and predictive methods to classify US for AI development. Data sets about this classification task do not exist. This study addresses this constraint through binary labeling (AI, not-AI) in the Mendeley dataset by a group of experts. The research evaluates three machine learning techniques for modeling the automatic classification of US.

TABLE 1. Summary of the literature review.

Related work	Approach	Used Techniques	Evaluation Metrics	Key Results
Subedi et al. (2021) [4]	Supervised classification to automate quality checking/analysis of user stories. Classifiers before and after balancing (smote,ros,rus,bt).	Data balancing techniques.	Accuracy, precisión.	No significant improvements in classification metrics.*0.79* accuracy, precision *0.86*, and recall *0.79.*
Yufeng Ma et al. (2021) [5]	Semantic recognition, segmentation for identification and composite stories from semantic analysis.	HanLP.	N/A	Better understanding and segmentation of user stories.
Dos Santos et al. (2024) [1]	Automatic user stories generation.	N-gram, GPT-3.	BLEU, ROUGE-N, BERTScore.	High quality in the user stories generation. *BERTScore=0.73.*
Kumar B. et al. (2023) [6]	Classification of functional/Non-functional requirements.	Random forest, XGBoost.	Accuracy.	RF y XGBoost: Accuracy *0.96*.
Tasnim et al. (2023) [7]	Automatic functional/Non-functional classification.	Assembling classifiers.	Accuracy.	Binary classification with *0.98* accuracy.
Levy et al. (2022) [8]	User stories instability metrics (changes after assigning to a developer)	Logistic regression, KNN, SVM.	Accuracy.	*KNN: 0.86* accuracy in instability detection.
Raharjana I. et al. (2021) [9]	Systematic literature review to capture the current state-of-the-art of NLP research on user stories .	Part-of-Speech (POS) tags are the most widely used NLP techniques, but semantic approaches (e.g., vector space models and machine learning) are starting to gain a place.	N/A	N/A
Siahaan D. et al. (2023) [10]	Extraction of user stories from natural language	Part-of-speech (POS) chunking, named entity recognition (NER), dependency parsing, WordNet, and BloomSoft.	Precision, Recall, and F-score	Improvements in identification and extraction of user stories.
Rahimi et al. (2021) [11]	Classification of software requirements (Functional or Non-Functional).	Deep learning suite.	Accuracy, F1 score, precision, recall.	*0.95* accuracy in binary classification and 0.93 in multi-class classification using Deep Learning.
Rahman y Zhu (2024) [12]	Automatic generation of user stories.	GPT-4.	Accuracy.	Average score of 4 out of 5 on the quality of user stories generated. Reduced burden on software engineers, increased productivity.
Kaur, Kamaljit & Kaur, Parminder (2024) [2]	Automation of requirements identification and classification.	Literature review of 60 research studies to automate the identification and classification of software requirements focused on identifying functional and non-functional requirements.	N/A	CNN and SVM achieve better results, but more collaboration is needed to overcome challenges in real-world environments.

This study raises several questions: Q1: How accurate are the binary classifications (AI, not-AI) of User Stories (US) performed by the proposed model? Q2: Which machine learning technique represents the lowest error in the binary classification of US? The answers to these questions are found in the results and conclusions of this document.

B. DATA PROTOCOL

To select an appropriate dataset and train an AI model that allows automatically identifying whether a user story is implementable using AI, a search for both public and private datasets of software projects with their respective US was carried out. After this search, the Mendeley repository Dalpiaz Fabiano[1] [13] was selected as the dataset. The dataset is a collection of 22 projects and 1680 US that represent the requirements of these software projects. This repository was chosen due to the diversity of projects and the quality of the US descriptions.

[1] https://data.mendeley.com/datasets/7zbk8zsd8y/1

1) DATASET PREPARATION BY EXPERTS

The dataset preparation for training the AI model was carried out following a systematic process. First, a team of experts in software development using agile methodologies and AI was formed. The experts were provided with a clear vision of the labeling project's objective and the importance of accurately determining which US are implementable using AI. This clarity in communication is essential to ensure all experts are aligned. Once the purpose was understood, the group of experts agreed on specific criteria to apply the manual classification process of the US. These criteria were collaboratively defined, considering the experts' experience. The agreed criteria ensure consistency and objectivity in the classification process.

Each US has a structure defined by: "the user, the most explicit description of the functionality possible." An example of a US is: "As a Broker user, I want to Upload and Validate the error message to have accurate text." In this case, the group of experts classified the US as not-AI. However, the US: "As a user, I want to be able to view a map display of the special waste drop-off sites around my area," was classified as AI by the experts, meaning it is susceptible to AI implementation.

The labeling was done in a group, reaching a consensus on each user story in each project, determining whether it is susceptible to being developed with AI. Thus, each user story was evaluated and assigned a value of 1 if it was considered suitable for AI implementation (in the document, this label was named AI), and a value of 0 if it was not (in the document, this label was named not-AI). In cases of ambiguity regarding the classification of a US, the topic was researched, then discussed, and a consensus was sought to make an informed decision. This approach aimed to ensure the accuracy and reliability of the classified dataset.

The experts applied *50* criteria, grouped into eight key categories: data handling and analysis, task automation, prediction and decision-making, advanced user interaction, natural language processing (NLP), image and video processing, integration and adaptation, and security and anomaly detection. Each US was evaluated based on these groups of criteria to determine its suitability for AI implementation. See appendix B.

Regarding how decisions were made based on the structure of the US, the classification process focused on identifying characteristics aligned with AI capabilities. Stories involving complex data handling, repetitive tasks, or advanced user interaction were classified as suitable for AI. This update provides deeper context about the reasoning behind the classification and its connection to the development of the machine learning models.

Following these steps, a manually classified repository of US was obtained. The manual classification of these US resulted in a dataset of *1680* US and *22* projects, in which the classes are unbalanced, with *654* AI-US and *1026* not-AI. Figure 1 presents the classification of US obtained by projects and the totals.

2) DATA SPLITTING

Given that out of a total of 1680 US, 60% are not-AI, a stratified class sampling technique was used to balance the dataset and separate the data (user stories) into training and testing sets. The data for training the model (training data: 1046) and for evaluating its performance (test data: 262) were divided accordingly. In this sense, the balanced training dataset consists of 523 US for each case, AI and not-AI. Similarly, 131 were used as the test set for both AI and not-AI, respectively.

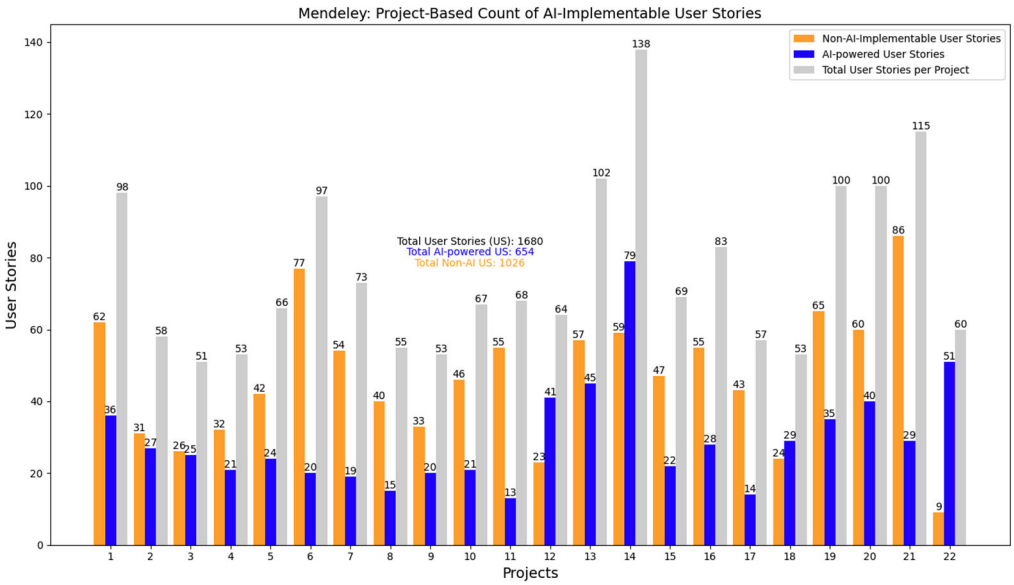


FIGURE 1. Distribution of manually classified US by experts from the Mendeley repository.

C. PROPOSED MODEL

To develop the model, the CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology proposed by Schröer was applied [14], [15]. Similarly, the literature reviewed was considered for the model development, where different works related to the use of machine learning and US were found. However, no work was found that performs the machine learning task proposed in this research. In this sense, a classification model capable of processing the text of the US descriptions and distinguishing those that belong to US that can be developed with AI from those that cannot is proposed. Three techniques commonly found in the state of the art are used to build a binary classifier that identifies US implementable with AI. Thus, the proposed model explores logistic regression, nearest neighbors, and support vector machines. This work will evaluate the performance metrics for three binary classification models of US developable with AI. The metrics evaluated are accuracy, loss (Log-Likelihood Loss), precision, recall, F1 score, area under the ROC curve, and specificity. For the definition and formula of each of these metrics, see appendix A.

The proposed classification model uses the description of the US to identify intrinsic patterns and relationships. The model is optimized to maximize performance quality through advanced text processing techniques and a regularization approach to improve model accuracy and generalization. In this way, textual features are transformed using various extraction techniques. These techniques include converting text to uppercase, segmenting into words and characters, applying TF-IDF (Term Frequency-Inverse Document Frequency), and using word vectors. These methods capture both the structure and meaning of the text, enhancing the model's ability to identify relevant patterns and make accurate predictions.

The proposed model was implemented for binary classification techniques: logistic regression, k-nearest neighbors, and support vector machines [9]. These methods have consistently demonstrated their effectiveness in binary classification tasks, providing an optimal balance between accuracy, robustness, and the ability to handle large volumes of data. Logistic regression is a widely used machine learning method for binary classification tasks, where the goal is to predict the probability that a data point belongs to one of two classes. Its popularity in this field is due to its simplicity in understanding and programming. Despite its simplicity, logistic regression offers a high level of accuracy in binary classification tasks for linearly separable datasets [8]. The K-Nearest Neighbors (KNN) binary classification model is useful in this context due to its effectiveness in determining whether a User Story (US) is a candidate for implementation with AI techniques. By comparing each US with its nearest neighbors in the feature space [9], KNN can leverage the similarities between US to make accurate predictions. Finally, the Support Vector Machine (SVM) binary classification model is useful in this context due to its ability to efficiently handle large feature spaces and its robustness in classifying complex data [8].

For the training process, an 80% data split for training is used, equivalent to 1256 US, and the remaining 20% for testing, equivalent to 314 US. During the hyperparameter tuning of the models, regularization is used in the case of logistic regression. In the case of nearest neighbors, DistributionSmoothing and KDtree are used as methods for the nearest neighbors search. Finally, in the case of support vector machine, the kernel type is Polynomial, and the polynomial degree is 2. Despite the imbalanced nature of the data, the data was balanced using stratified class sampling. The models' effectiveness in classifying User Stories is evaluated through standard classifier performance evaluation metrics. This tests the feasibility of assisted generation of the Smart Product Backlog.

D. CLASSIFICATION ALGORITHMS

The supervised machine learning techniques used in this research work are explained below.

1) TEXT PROCESSING FOR FEATURE EXTRACTION

Text preprocessing for feature extraction in this model is implemented using various techniques that transform text into numerical representations suitable for machine learning. The process begins by applying the 'ToUpperCase' function, which converts all text to uppercase to standardize the format and reduce variability caused by capitalization differences. Next, the text is segmented into words, making it easier to analyze each term individually. To capture the relative importance of words within the documents, the TFIDF method is applied, which weighs each term based on its frequency in the document and its rarity across the corpus, highlighting the most representative terms. Additionally, word vector representations are integrated, capturing semantic relationships between words in a high-dimensional vector space, enabling the model to recognize word similarities. Finally, character-level segmentation is performed, which is useful for analyzing prefixes, suffixes, or repetitive patterns within words.

2) LOGISTIC REGRESSION

Logistic regression is a widely used statistical technique for binary classification in machine learning. The model calculates the probability that an instance belongs to one of two classes, using a sigmoid function to convert a linear combination of features into a probability between 0 and 1. This technique is particularly effective for problems where the relationships between independent variables and the target class are linear. In the context of classifying user stories (US) suitable for implementation with artificial intelligence, logistic regression can be useful for predicting whether a story is appropriate for AI-based solutions by evaluating factors such as task complexity, available data, and required processing. Its simplicity and high level of interpretability allow development teams to better understand the criteria that make a user story suitable for AI implementation.

3) K-NEAREST NEIGHBORS (K-NN)

The KNN algorithm classifies instances based on their similarity to their nearest neighbors in the feature space. In the context of classifying user stories for AI implementation, KNN can identify stories that are similar to a new one based on its description, complexity, and task characteristics. For example, if several previous user stories involving natural language processing or computer vision were successfully implemented with AI, the KNN model can classify new user stories sharing those characteristics as candidates for AI. This technique is useful when historical examples of previously classified user stories are available, as it leverages existing patterns for new predictions. The K-Nearest Neighbors (k-NN) algorithm in this context uses advanced configurations to enhance its performance in classifying user stories implementable with artificial intelligence. In this case, the nearest neighbor method is set to DistributionSmoothing $= 2$ and NearestMethod is KDTree, which optimizes the search for nearest neighbors using a KD tree to improve efficiency in large feature spaces. The DistributionSmoothing $= 2$ parameter adjusts the smoothing of the neighbors distribution, providing greater robustness against data dispersion in the feature space. These configurations make KNN more efficient and precise when classifying user stories that share similarities with previously successful AI-based implementations, improving the model's ability to identify complex patterns and make more reliable predictions.

4) SUPPORT VECTOR MACHINES (SVM)

SVM is a powerful binary classification technique designed to maximize the separation between classes by creating an optimal hyperplane in the feature space. In classifying user stories implementable with artificial intelligence, SVMs are particularly useful when the features of user stories are not linearly separable, due to their ability to handle large feature sets and transform data into a higher-dimensional space using kernel functions. This is ideal in cases where user stories present complex or abstract characteristics, such as the integration of large-scale data analysis or deep learning systems, allowing precise identification of which stories can benefit from AI use. The Support Vector Machine (SVM) algorithm in this implementation uses advanced configurations to efficiently handle the classification of user stories implementable with artificial intelligence. The kernel type is specified as Kernel Type is Polynomial, meaning that the model uses a polynomial kernel to project the data into a higher-dimensional space, allowing for more accurate separation of non-linear classes. This kernel is especially useful when user stories exhibit complex characteristics and non-linear relationships that need to be captured. These configurations enable SVM to manage large volumes of data with multiple features, robustly identifying user stories that could benefit from AI implementation, particularly in scenarios where feature relationships are neither trivial nor linear.

E. MODEL ARCHITECTURE

Figure 2 presents the data flow, from the information sources Mendeley user stories to binary classification AI and not-AI. In this regard, it is possible to determine the predictive capacity of the classification model and its lowest error.

IV. RESULTS

In this work, we evaluated Q1: How accurate are the binary classifications (AI, not-AI) of User Stories (US) performed by the proposed model? and Q2: Which machine learning technique represents the lowest error in the binary classification of the US? In this sense, the aim is to recognize the viability of the models in the binary classification of User

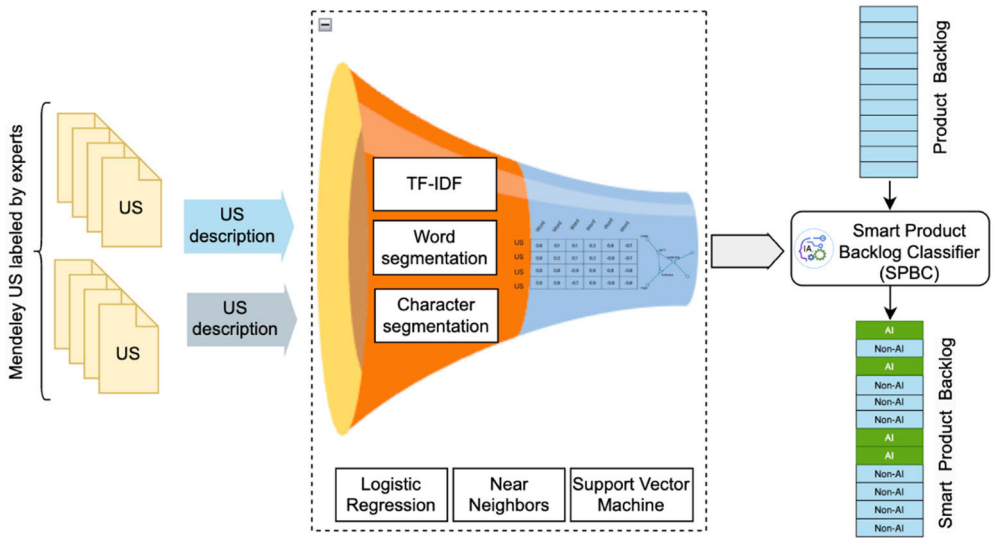


FIGURE 2. Smart product backlog classifier.

Stories (US) implementable with AI, as well as the potential of the assisted generation of the Smart Product Backlog.

To test the Smart Product Backlog Classifier (SPBC), three classification methods were used, and different metrics were calculated to understand the model's performance and effectiveness. The evaluated metrics are accuracy, loss, prediction accuracy, recall, F1 score, area under the ROC curve, and specificity. The table 1 presents the results of accuracy and Log-Likelihood Loss.

TABLE 2. SPBC model metrics.

Metric	Logistic Regression	Nearest Neighbors 1	SVM
Accuracy	**0.748092**	0.545802	**0.740458**
Loss (Log-Likelihood Loss)	-151.078	-181.224	-160.923

A. CLASSIFICATION USING LOGISTIC REGRESSION

In Figure 3, the results of logistic regression are shown. The model achieves an accuracy of 0.748, indicating that the model correctly classifies 74.8% of the user stories, which is significantly better than the baseline accuracy of 50.0%, a completely random classification.

Regarding the calculated metrics, the model has a Log-Likelihood Loss of −151.078, indicating a good fit of the model. The model's precision is good, with values of 0.740741 specifically for the not-AI class and 0.755906 for the AI class, suggesting that the model is more accurate in identifying stories susceptible to AI implementation. The recall shows values of 0.763359 for the not-AI class and 0.732824 for the AI class, indicating that the model is slightly better at correctly identifying user stories not susceptible to AI implementation. The F1 Score, which is a balanced metric between precision and recall, shows values of 0.75188 for the not-AI class and 0.744186 for the AI class, reinforcing the model's good ability to classify the two classes in a balanced way. The ROC-AUC, which measures the model's ability to distinguish between classes, is 0.802634 for both classes, indicating the model's good performance in terms of class separation. Finally, the specificity is 0.732824 for the not-AI class and 0.763359 for the AI class, showing that the model has a good ability to correctly identify stories not susceptible to AI implementation, see Table 3.

B. CLASSIFICATION USING NEAREST NEIGHBORS

In Figure 3, the results of the classification model using the Nearest Neighbors method are shown. The model achieves an accuracy of 0.54 (54.6%), which is only marginally better than flipping a coin (a model that randomly classifies the two classes). Regarding the calculated metrics, the Log-Likelihood Loss was −181.224, suggesting that the model does not fit the data as well as other models. The precision is moderate, with values of 0.529703 for the not-AI class and 0.6 for the AI class. This indicates that the model has higher precision in classifying stories susceptible to AI implementation but remains relatively low overall. The recall shows a significant disparity, with a value of 0.816794 for the not-AI class and 0.274809 for the AI class, indicating that the model is much better at identifying stories not susceptible to AI implementation but significantly fails to correctly identify those that are. The F1 Score is 0.642643 for the not-AI class and 0.376963 for the AI class, again highlighting the model's weakness in classifying the AI class. The ROC-AUC shows a value of 0.224462 for the not-AI class, while it is significantly better for the AI class at 0.867141, indicating that the model has limited ability to correctly discriminate between the classes. The specificity shows that the model has a low ability to correctly identify not-AI user stories, see Table 3.

C. CLASSIFICATION USING SUPPORT VECTOR MACHINE (SVM)

In Figure 3, the results of the classification model using Support Vector Machine (SVM) are shown. The model achieves an accuracy of 0.74, indicating that the model correctly classifies 74.0% of the user stories. Additionally, the Log-Likelihood Loss was −160.923, suggesting a good fit of the model to the data. The precision for the not-AI class is 0.72028 and 0.764706 for the AI class, indicating that the model is more precise in identifying stories susceptible to AI implementation. The recall shows values of 0.78626 for the not-AI class and 0.694656 for the AI class, indicating that the model has a better ability to correctly identify user stories not susceptible to AI implementation but is still effective in identifying those that are. The F1 Score is 0.751825 for the not-AI class and 0.728 for the AI class, reinforcing the model's balanced ability to effectively classify both classes. The ROC-AUC, which measures the model's ability to distinguish between classes, is 0.77781 for both classes, indicating good performance of the model in terms of class separation. The specificity shows that the model has a good ability to correctly identify not-AI user stories (0.694656) and a good ability to identify AI stories (0.78626), see Table 3.

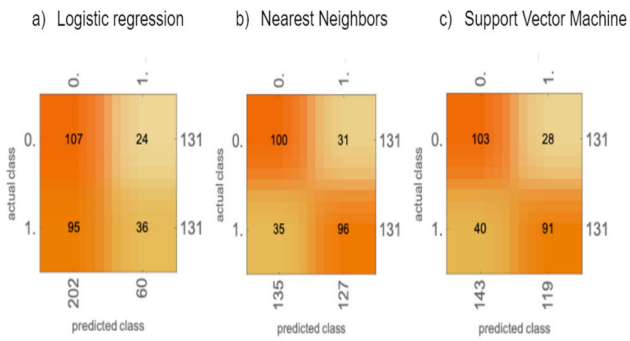


FIGURE 3. Results of classification models confusion matrix.

D. RESULTS EVALUATION

When comparing the results of the three user story classification methods, it is observed that the Logistic Regression and

TABLE 3. Performance evaluation metrics of RL, VMC, and SVM models.

Metric	Logistic Regression		Nearest Neighbors		SVM	
	No-IA	IA	No-IA	IA	No-IA	IA
Precision	0.740741	**0.755906**	0.529703	**0.6**	0.72028	**0.764706**
Recall	**0.763359**	0.732824	**0.816794**	0.274809	**0.78626**	0.694656
F1 Score	**0.75188**	0.744186	**0.642643**	0.376963	**0.751825**	0.728
ROC-AUC	**0.802634**	**0.802634**	0.224462	**0.867141**	**0.77781**	**0.77781**
Specificity	0.732824	**0.763359**	0.274809	**0.816794**	0.694656	**0.78626**

Support Vector Machine (SVM) models exhibit significantly superior performance compared to the Nearest Neighbors method. Both Logistic Regression and SVM show high accuracy, with values of 0.748092 and 0.740458 respectively, and relatively low log-likelihood loss, indicating a good fit of the model. Additionally, both precision and recall are balanced for both classes in these two models, with F1 Scores and ROC-AUC values reinforcing their effective discrimination ability between user stories susceptible and not susceptible to AI implementation. In this sense, it is a first response to Q1.

In particular, the SVM model slightly stands out in terms of recall for the not-AI class (0.786260) and precision for the AI class (0.764706), suggesting a better ability to identify correctly labeled AI stories. In contrast, the Nearest Neighbors method shows inferior performance, with lower precision and greater disparity in precision and recall metrics between classes, making it less reliable for this classification task

Finally, the SVM method stands out as effective for classifying user stories implementable with AI, offering robust balance and consistent performance across all evaluated metrics. Table 3 presents a comparative performance metric of the three models.

In this work, the accuracy and other performance metrics of the binary classifications (AI, not-AI) of User Stories (US) were evaluated. The results presented in Tables 2 and 3 show that logistic regression and SVM techniques exhibit better overall performance. This work thus establishes the feasibility of automatically constructing a SPBL.

This work presents a dataset that allows further research in this field, and the implementation of the models was carried out in Wolfram Mathematica. The dataset and codes are available in the following MauricioGaona GitHub repository[2] [16].

V. DISCUSSION

It is important to highlight that the literature review found works that perform automatic classification of user stories ([4], [5], [9]), classifications focused on automatically identifying functional and non-functional user stories ([6], [7], [11]), and related classifiers using different machine learning techniques ([8]). Finally, there are related works with MLSs for generating user stories ([1], [10], [12]). This research evaluates the prediction of Non-AI and AI classes from the US. Although there are no similar works in the literature review, the state of the art was considered to define the techniques to be implemented as a first approach to the problem.

To the best of our knowledge, there is currently no existing model or method specifically designed to classify user stories as implementable with AI or not. While various approaches have been explored for classifying software requirements and user stories based on different criteria, none have focused explicitly on identifying those that are feasible for AI implementation. This study addresses that gap by proposing a model that leverages machine learning techniques to automate the classification of user stories in the context of AI development.None of the related work focuses on classifying user stories as implementable with AI or not, as they generally address different aspects of software requirements classification. Therefore, it is not possible to directly compare our results from the logistic regression and SVM models with the findings in the references. Our study addresses a unique classification task, which fills a gap not covered by the found literature.

The objectives of this study include understanding the behavior of classification techniques in the problem of separating US between Non-AI and AI, contributing to the requirements elicitation process, and helping to build the SPB. This, in turn, aims to offer tools for agile software development processes. Logistic regression, nearest neighbors, and SVM were used, as shown in tables 2 and 3. Thanks to the F1-Score and AUC-ROC, it was established that the techniques offering the best performance are logistic regression and SVM.

Regarding the construction of the dataset, it can be observed that the expert classification process allowed the construction of an automatic classifier with an AUC-ROC performance between 0.77 and 0.8 and an F1-score close to 0.75. Thus, both the proposed dataset and the classification models allowed the construction of a tool that supports SPB decision-making. This suggests that logistic regression and SVM models are more suitable for automatically classifying the SPB.

The success rate for classifying AI-implementable US is over 76%, indicating that the model correctly classifies approximately 8 out of 10 AI-implementable US.

[2]https://github.com/mauriciogaona/Smart-Product-Backlog

The associated error in this classification would be 24%, meaning approximately 2 out of 10 US would be misclassified.

We discuss how automating the classification of user stories suitable for implementation with artificial intelligence can streamline the prioritization process, reduce manual workload, and improve decision-making in the construction of the Smart Product Backlog. We believe this addition significantly enhances the practical value of our work and hope it adequately addresses your concern.

The Gherkin template, widely utilized in Behavior-Driven Development (BDD), is also pertinent and could provide valuable insights into the applicability and effectiveness of our model across various real-world scenarios [17]. Future research could investigate the application of our model to user stories written in other formats, such as Gherkin. We believe this would further enhance the generalizability of our findings and offer additional value to the broader community.

One of the main limitations of this project is the dependence on high-quality labeled data to train the classification models. The manual labeling process of user stories (US) requires significant time and effort from experts. Additionally, subjectivity in labeling can introduce inconsistencies, affecting the model's accuracy. Similarly, it is necessary to test the model in different application domains. Models trained on a specific dataset may not perform as well in other contexts or industries with different types of US and requirements. This may require context-specific adjustments.

VI. CONCLUSION

The classification and analysis of requirements and US are resource-intensive, involving multiple meetings with stakeholders. Automating this process is expected to alleviate the additional burden on software engineers, enabling them to focus on higher-priority tasks and boosting productivity.

The goal is to optimize the Smart Product Backlog (SPB) by automating the identification of US that can be developed with AI, thereby facilitating the strategic development of intelligent applications.

The study introduces a dataset created specifically for this project, available for future research, and implements the models using Wolfram Mathematica.

The logistic regression and SVM models are more suitable for automatically classifying the SPB, separating US between Non-AI and AI. The model correctly classifies approximately 8 out of 10 AI-implementable US. The associated error in this classification would be 24%, meaning approximately 2 out of 10 US would be misclassified.

The development of a Smart Product Backlog, which integrates AI-based functionalities, would improve the system's architecture and provide a more comprehensive and strategic view of an application's capabilities. This, in turn, would allow development teams to design efficient, scalable, and higher-performing systems. Consequently, it is relevant to contrast the manual classification of US due to the inherent subjectivity of the classification process, among other previously described problems. Thus, the automatic classification of US with AI techniques optimizes resource management by reducing time. This automation minimizes error, achieving greater consistency and precision, which is crucial for large development projects with hundreds of US. This work provides a valuable contribution as it seeks to determine the confidence level between different automatic classification models. Therefore, it offers important information for the use of US data by academics and researchers in the context of requirements elicitation.

For future work, it is planned first to build models that surpass 74% accuracy. This precision, although relatively high, still implies that one in four US could be incorrectly classified. It is proposed to use more advanced learning techniques, such as deep neural networks and ensemble models (combining predictions from different models), that can capture more complex patterns in the data. It would also be beneficial to expand and diversify the training dataset, incorporating US from different domains and contexts to improve the model's generalization ability. Additionally, implementing AI-assisted automatic labeling methods could reduce the manual effort in data preparation. Finally, developing tools and processes to integrate the model more smoothly into existing workflows could facilitate its adoption and maximize its impact on the assisted construction of the SPB and the agile development of intelligent applications.

APPENDIX A
METRICS DEFINITION

See Table 4.

APPENDIX B
CLASIFICATION CRITERIA OF AI OR NOT AI BY EXPERTS

The following 50 criteria were used by AI experts to classify user stories from the Mendeley repository as implementable with artificial intelligence. These criteria are distributed across 8 distinct groups—Data Management and Analytics, Task Automation, Prediction and Decision Making, User Interaction and Personalization, Natural Language Processing (NLP), Image and Video Processing, Integration and Adaptation, and Security and Anomaly Detection. This classification framework enabled the experts to evaluate the potential for AI implementation based on each user story's alignment with capabilities like data analysis, automation, prediction, and user interaction.

1. Data Management and Analytics
    1. Handling large volumes of data
    2. Pattern recognition capabilities
    3. Automatic classification of data
    4. Sentiment analysis
    5. Fraud detection and prevention
    6. Real-time data processing
    7. Data aggregation and summarization
    8. Identifying trends and anomalies in data
    9. Multidimensional data analysis
    10. Data quality management

TABLE 4.

Metric	Definition	Formula
Accuracy	Accuracy is a commonly used metric to evaluate the performance of a classification model. It represents the proportion of correct predictions made by the model	*Accuracy = (True Positives + True Negatives) / (Total Samples)*
Loss	Loss is a measure of the error between the predicted values and the actual values. It is used to optimize the training process of a machine learning model. In binary classification using logistic regression, the loss function is often cross-entropy loss. **y_pred** refers to the values predicted by the machine learning model and **y_true** represents the actual or true values for the given input data.	*Loss = - (sum(y_true x log(y_pred)) + (1 - y_true) x log(1 - y_pred))*
Precision	Precision measures the proportion of positive predictions that are actually correct. It is calculated as the number of True Positives divided by the total number of positive predictions (True Positives + False Positives).	*Precision = True Positives / (True Positives + False Positives)*
Recall	Recall measures the proportion of actual positives that are correctly identified by the model. It is calculated as the number of True Positives divided by the total number of actual positives (True Positives + False Negatives).	*Recall = True Positives / (True Positives + False Negatives)*
F1 Score	The F1 score is a harmonic mean of precision and recall, providing a balanced measure of both aspects of a model's performance.	*F1 Score = 2 x (Precision x Recall) / (Precision + Recall)*
Area Under ROC curve	The AUC is calculated from the ROC curve, which is a plot of the true positive rate (Recall) against the false positive rate FP=(FP+TNFP) for different decision thresholds. It does not have a simple direct formula but can be calculated using methods such as the trapezoidal rule.	*FP = Falsos Positivos* *FN = Falsos Negativos*
Specificity	Specificity measures the proportion of actual negatives that are correctly identified by the model. It is calculated as the number of True Negatives divided by the total number of actual negatives (True Negatives + False Positives).	*Specificity = True Negatives / (True Negatives + False Positives)*

2. Task Automation
    11. Automation of routine processes
    12. Automated technical support
    13. Content management automation
    14. Automatic information generation and reporting
    15. Automating repetitive data entry tasks
    16. Task assignment optimization through AI
    17. Monitoring of system performance through automation
    18. Automated compliance and regulatory checks
3. Prediction and Decision Making
    19. Predictive analytics for future outcomes
    20. Preventive monitoring and alerting
    21. Resource and process optimization through AI
    22. AI-assisted decision-making processes
    23. Risk assessment and mitigation through AI
    24. Predicting user behaviors and actions
    25. Real-time decision support systems
4. User Interaction and Personalization
    26. Personalized content recommendation
    27. Customizing user experiences through AI
    28. Advanced user interaction via chatbots and virtual assistants
    29. Voice-based user interaction
    30. Real-time feedback and adaptation to user needs
5. Natural Language Processing (NLP)
    31. Understanding user-generated text
    32. Automatic generation of text
    33. Text summarization and abstraction
    34. Language translation through AI
    35. Automatic response generation in natural language
    36. Contextual understanding of conversations
6. Image and Video Processing
    37. Object recognition in images
    38. Real-time video analysis
    39. Facial recognition for security or personalization
    40. Scene understanding and annotation
    41. Detecting and classifying anomalies in video footage
    42. Optical character recognition (OCR) from images
7. Integration and Adaptation
    43. Seamless integration with external systems
    44. Adaptive learning based on new data
    45. Continuous system improvement
    46. Cross-platform integration and support
    47. Flexible scaling based on usage patterns
8. Security and Anomaly Detection
    48. Detection of unusual or suspicious behavior
    49. Threat detection and prevention
    50. Predictive threat intelligence

REFERENCES

[1] C. A. D. Santos, K. Bouchard, and F. Petrillo, "AI-driven user story generation," in *Proc. Int. Conf. Artif. Intell., Comput., Data Sci. Appl. (ACDSA)*, Victoria, Seychelles, Feb. 2024, pp. 1–6, doi: 10.1109/acdsa59508.2024.10467677.

[2] K. Kaur and P. Kaur, "The application of AI techniques in requirements classification: A systematic mapping," *Artif. Intell. Rev.*, vol. 57, no. 3, pp. 1–48, 2024, doi: 10.1007/s10462-023-10667-1.

[3] B. Kitchenham, "Procedures for performing systematic reviews," Softw. Eng. Group, Dept. Comput. Sci., Keele Univ., Keele, U.K., Tech. Rep. TR/SE-0401,2004.

[4] I. M. Subedi, M. Singh, V. Ramasamy, and G. S. Walia, "Classification of testable and valuable user stories by using supervised machine learning classifiers," in *Proc. IEEE Int. Symp. Softw. Rel. Eng. Workshops (ISSREW)*, Wuhan, China, Oct. 2021, pp. 409–414, doi: 10.1109/ISSREW53611.2021.00111.

[5] Y. Ma, Y. Dou, M. Wang, Y. Wang, M. Jiang, and Y. Lu, "Automatic recognition and segmentation of composite user stories based on semantic analysis," in *Proc. 5th Int. Conf. Comput. Sci. Appl. Eng.*, vol. 58, New York, NY, USA, Oct. 2021, pp. 1–7, doi: 10.1145/3487075.3487089.

[6] B. Kumar, U. K. Tiwari, and D. C. Dobhal, "Classification of NFR based importance level of user story in agile software development," in *Proc. 9th Int. Conf. Signal Process. Commun. (ICSC)*, vol. 2, Noida, India, Dec. 2023, pp. 264–268, doi: 10.1109/icsc60394.2023.10441284.

[7] A. Tasnim, N. Akhter, and K. M. A. Ali, "A fine tuned ensemble approach to classify requirement from user story," in *Proc. 26th Int. Conf. Comput. Inf. Technol. (ICCIT)*, Cox's Bazar, Bangladesh, Dec. 2023, pp. 1–6, doi: 10.1109/iccit60459.2023.10441143.

[8] Y. Levy, R. Stern, A. Sturm, A. Mordoch, and Y. Bitan, "An impact-driven approach to predict user stories instability," *Requirement Eng.*, vol. 27, no. 2, pp. 231–248, Jun. 2022, doi: 10.1007/s00766-022-00372-w.

[9] I. K. Raharjana, D. Siahaan, and C. Fatichah, "User stories and natural language processing: A systematic literature review," *IEEE Access*, vol. 9, pp. 53811–53826, 2021, doi: 10.1109/ACCESS.2021.3070606.

[10] D. Siahaan, I. K. Raharjana, and C. Fatichah, "User story extraction from natural language for requirements elicitation: Identify software-related information from online news," *Inf. Softw. Technol.*, vol. 158, Jun. 2023, Art. no. 107195, doi: 10.1016/j.infsof.2023.107195.

[11] N. Rahimi, F. Eassa, and L. Elrefaei, "One- and two-phase software requirement classification using ensemble deep learning," *Entropy*, vol. 23, no. 10, p. 1264, Sep. 2021, doi: 10.3390/e23101264.

[12] T. Rahman and Y. Zhu, "Automated user story generation with test case specification using large language model," 2024, *arXiv:2404.01558*.

[13] D. Fabiano, 2018, "Requirements data sets (user stories)," *Mendeley Data, V1*, doi: 10.17632/7zbk8zsd8y.1.

[14] C. Schröer, F. Kruse, and J. M. Gómez, "A systematic literature review on applying CRISP-DM process model," *Proc. Comput. Sci.*, vol. 181, pp. 526–534, Jan. 2021.

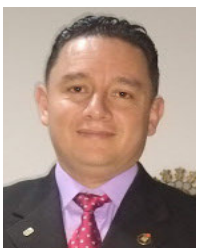
[15] J. S. Saltz, "CRISP-DM for data science: Strengths, weaknesses and potential next steps," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Orlando, FL, USA, Dec. 2021, pp. 2337–2344, doi: 10.1109/BigData52589.2021.9671634.

[16] M. Gaona. *Github Repository*. Accessed: Aug. 15, 2024. [Online]. Available: https://github.com/mauriciogaona/Smart-Product-Backlog

[17] D. Mateus, D. S. da Silveira, and J. Araújo, "A systematic approach to derive user stories and gherkin scenarios from BPMN models," in *Business Modeling and Software Design*, im Advances in Multimedia, Software Engineering and Computing, vol. 483, B. Shishkov, Ed., Cham, Switzerland: Springer, doi: 10.1007/978-3-031-36757-1_15.

MAURICIO GAONA-CUEVAS received the Ph.D. degree in computer science from the University of Massachusetts Lowell, in 1998. He is currently a Full Professor with the School of Computer Systems, Universidad del Valle, Cali, Colombia. Additionally, he leads the Software Engineering Group, Universidad del Valle. His research interests include smart applications, software engineering, and web development. His main research activities focus on AI-augmented software engineering.

VÍCTOR BUCHELI GUERRERO received the degree in systems engineering, the master's degree in systems engineering and computation, and the Ph.D. degree in engineering, in 2013. He is currently a Full Professor with the School of Computer Systems, Universidad del Valle, Cali, Colombia. He also leads the Artificial Intelligence Group, Universidad del Valle. He has authored or co-authored over 30 publications in international journals and conferences. His research interests include artificial intelligence, network science, and knowledge management. His research involves developing and applying complex networks and machine-learning models. He holds a Senior Research Rank awarded by MinCiencias.



FREDY H. VERA-RIVERA received the degree in systems engineering and the master's degree in engineering (computer science) from Universidad Industrial de Santander, Bucaramanga, Colombia, in 2005 and 2009, respectively, and the Ph.D. degree in engineering and computer science from Universidad del Valle, Cali, Colombia, in 2021.

From 2005 to 2013, he was a Web Developer and a Technical Leader in software companies in Bucaramanga, Colombia. He is currently an Assistant Professor with Universidad Francisco de Paula Santander, Cúcuta, Colombia. His research interests include digital transformation, microservice architectures, cloud computing, component-based software, artificial intelligence, agile software development, and web development.